ASSIGNMENT-5 SOLUTIONS: Asymmetric Cryptography

COURSE	SUBJECT CODE	SUBJECT TITLE	TOPIC
BCA	**BCA309T-AS05**	**Cryptography & Network Security**	**Introduction to Asymmetric Cryptography**

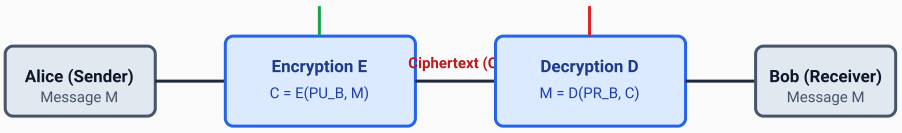
Q1. Overview and Principles of Public-Key Cryptography [10 Marks] LONG ANSWER

Basic Principles: Asymmetric (public-key) cryptography uses a mathematically linked *key pair* consisting of a **Public Key** (PU) that is published openly, and a **Private Key** (PR) that is kept strictly secret by its owner.

- **Public Key (PU_B):** Distributed publicly. Anyone can use Bob's public key to encrypt a message intended for Bob or to verify Bob's digital signature.
- **Private Key (PR_B):** Kept secret by Bob. Only Bob can decrypt messages encrypted with his public key or generate a digital signature under his name.

Achieving Security Goals:

- **Confidentiality:** Alice encrypts message M using Bob's public key: $C = E(PU_B, M)$. Only Bob can decrypt it using PR_B: $M = D(PR_B, C)$.
- **Authentication & Non-Repudiation:** Alice signs M using her private key: $S = E(PR_A, M)$. Anyone can verify using PU_A: $M = D(PU_A, S)$. Because only Alice possesses PR_A, this authenticates Alice as the sender and prevents her from denying (repudiating) creating the message.



Q2. Symmetric vs Asymmetric Cryptography Comparison [8 Marks] TABULAR

Comparison Parameter	Symmetric Cryptography	Asymmetric Cryptography
Number of Keys	Single shared secret key (K) used for both encryption & decryption.	Pair of asymmetric keys: Public Key (PU) and Private Key (PR).
Key Distribution	Extremely difficult; requires secure out-of-band channel before communication.	Simple; public key distributed freely, private key never shared.
Execution Speed	Very fast (simple XOR, shift, substitution operations).	Slow (100–1000x slower; complex modular exponentiations).
Key Length	Smaller for high security (e.g., AES-128, AES-256 bits).	Substantially larger (e.g., RSA-2048, RSA-4096 bits for equivalent security).
Scalability (n users)	Poor scalability; requires $\frac{n(n-1)}{2}$ secret keys across network.	High scalability; requires only $2n$ total keys (1 pair per user).
Typical Applications	Bulk data encryption (file storage, TLS application payload).	Key exchange (Diffie-Hellman), Digital Signatures, TLS handshakes.

Q3. Main Building Blocks of Public-Key Cryptography [10 Marks] LONG ANSWER

1. Trapdoor One-Way Functions: A mathematical function $f(x)$ that is easy to compute in one direction $y = f(x)$, but computationally infeasible to invert $x = f^{-1}(y)$ unless a special secret piece of knowledge (the "trapdoor") is known. In RSA, multiplying two prime numbers $p \times q = n$ is easy, but factoring n back into p and q is extremely hard without the private key exponent d.

2. Diffie-Hellman Key Exchange: A cryptographic protocol allowing two untrusted parties (Alice and Bob) to establish a shared secret key over an insecure channel without transmitting the secret itself.
• *Mechanism:* Agree on public parameters (p, g). Alice picks private a, sends $A = g^a \pmod p$. Bob picks private b, sends $B = g^b \pmod p$. Both compute shared key $K = B^a \pmod p = A^b \pmod p = g^{ab} \pmod p$.

3. Digital Signatures: A scheme providing authenticity, integrity, and non-repudiation. The sender hashes the message $H(M)$ and encrypts the hash using their private key $S = E(PR_{sender}, H(M))$. The recipient decrypts S using PU_{sender} and verifies it against the freshly calculated $H(M)$.

4. Public Key Infrastructure (PKI) & Digital Certificates: PKI binds public keys to verified identities (users/domains) via standard **X.509 Digital Certificates**. A trusted **Certificate Authority (CA)** digitally signs the certificate, preventing Man-in-the-Middle (MitM) identity spoofing.

Q4. Historical Development of Public-Key Cryptography [10 Marks] LONG ANSWER

(a) Key Distribution Problem: Prior to the 1970s, symmetric ciphers required physical key distribution (couriers, secure mail). For a large network of n users, managing $O(n^2)$ secret keys was logistically impossible.

(b) Classified Work at GCHQ (1969–1973): James Ellis first conceptualized "non-secret encryption." In 1973, Clifford Cocks formulated an asymmetric algorithm equivalent to RSA, and Malcolm Williamson developed the key exchange scheme later known as Diffie-Hellman. These discoveries remained classified by UK Intelligence until 1997.

(c) Diffie-Hellman Breakthrough (1976): Whitfield Diffie and Martin Hellman published their landmark paper *"New Directions in Cryptography,"* introducing public-key exchange concepts and solving key distribution publicly for the first time.

(d) Invention of RSA (1977): Ron Rivest, Adi Shamir, and Leonard Adleman created the first practical public-key cryptosystem capable of both general encryption and digital signatures, using prime factorization hardness.

(e) Subsequent Innovations (ElGamal & ECC):
• *ElGamal (1985):* Taher ElGamal proposed an asymmetric scheme based on Discrete Logarithm hardness, forming the basis for DSA.
• *Elliptic Curve Cryptography (ECC, 1985):* Koblitz and Miller introduced ECC, offering equal security to RSA at drastically smaller key sizes (e.g., 256-bit ECC $\approx$ 3072-bit RSA).

Q5. Advantages, Disadvantages & Hybrid Cryptosystems [8 Marks] LONG ANSWER

Advantages of Asymmetric Cryptography:
1. Eliminates secret key distribution problems over untrusted networks.
2. Scalable ($2n$ keys vs $n(n-1)/2$).
3. Enables digital signatures for non-repudiation and authentication.

Disadvantages:
1. Extremely computationally intensive (~1000x slower than AES).
2. Requires large key sizes, increasing storage and bandwidth overhead.
3. Vulnerable to Man-in-the-Middle attacks if public keys are not authenticated via PKI.

Hybrid Cryptosystems (e.g., TLS / HTTPS): Modern real-world protocols combine both systems to achieve maximum security and high performance. Asymmetric cryptography is used during setup to authenticate parties and exchange a fresh symmetric session key; then symmetric cryptography (AES) encrypts high-volume payload data.

HTTPS Handshake Example: Your browser uses *RSA / ECDHE* (Asymmetric) to verify the server certificate and establish a secret 256-bit key. Once agreed, browser and server switch to *AES-256-GCM* (Symmetric) to transfer web pages instantly.

Q6. Detailed RSA Algorithm, Key Generation & Numerical Example [12 Marks] ALGORITHM & PROOF

Step-by-Step Key Generation Process:

1. Select two distinct, large prime numbers p and q.
2. Compute modulus $n = p \times q$. The length of n in bits is the RSA key length.
3. Compute Euler's Totient function: $\phi(n) = (p - 1)(q - 1)$.
4. Select an integer public exponent e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$ (coprime).
5. Calculate secret private exponent d such that $d \times e \equiv 1 \pmod{\phi(n)}$, i.e., $d = e^{-1} \pmod{\phi(n)}$ using Extended Euclidean Algorithm.
6. **Public Key:** $PU = (e, n)$; **Private Key:** $PR = (d, n)$.

Encryption & Decryption Formulas:

• **Encryption:** Ciphertext $C = M^e \pmod n$ (where message $M < n$).
• **Decryption:** Plaintext $M = C^d \pmod n$.

Mathematical Proof of Correctness:

Decryption evaluates $C^d \pmod n = (M^e)^d \pmod n = M^{ed} \pmod n$. Since $ed \equiv 1 \pmod{\phi(n)}$, we can write $ed = k \cdot \phi(n) + 1$. By Euler's Theorem ($M^{\phi(n)} \equiv 1 \pmod n$ when $\gcd(M,n)=1$), $M^{k \cdot \phi(n) + 1} = (M^{\phi(n)})^k \cdot M^1 \equiv (1)^k \cdot M \equiv M \pmod n$.

Step-by-Step Numerical Example:

1. Choose $p = 61$, $q = 53$.
2. Compute $n = 61 \times 53 = 3233$.
3. Compute $\phi(n) = (61 - 1)(53 - 1) = 60 \times 52 = 3120$.
4. Select $e = 17$ (since $\gcd(17, 3120) = 1$).
5. Compute $d = 17^{-1} \pmod{3120} = 2753$ (because $17 \times 2753 = 46801 = 15 \times 3120 + 1$).
6. **Public Key:** $(17, 3233)$, **Private Key:** $(2753, 3233)$.
7. *Encrypt Message $M = 65$:* $C = 65^{17} \pmod{3233} = 2790$.
8. *Decrypt Ciphertext $C = 2790$:* $M = 2790^{2753} \pmod{3233} = 65$. (Original message recovered!)

Q7. RSA Key Sizes, ECC Equivalent Strength & Quantum Threat [10 Marks] LONG ANSWER

RSA Key Size & Security Basis: The RSA key size is defined by the bit-length of modulus n. Security depends entirely on the computational difficulty of factoring n into p and q. Using General Number Field Sieve (GNFS), factoring scales exponentially with bit length.

Recommended RSA Key Sizes vs. ECC Equivalents:

RSA Key Size	ECC Key Size	Security Level (Bits)	Current Status & Recommendation
1024 bits	160 bits	80 bits	**Deprecated / Unsecure.** Factorable by nation-states.
2048 bits	224–256 bits	112 bits	**Current Minimum Standard.** Secure for current commercial use.
3072 bits	256 bits	128 bits	**Recommended High Security.** Secure beyond 2030.
4096 bits	384 bits	192 bits	**Ultra-Secure.** High computational overhead; legacy military.

Quantum Threat (Shor's Algorithm): Quantum computers running Shor's Algorithm solve prime factorization in polynomial time $O((\log n)^3)$. A quantum computer with ~4000 logical qubits will instantly break 2048-bit RSA, driving the transition to Post-Quantum Cryptography (PQC).

Q8. Security Considerations, Known Attacks & RSA Padding [12 Marks] LONG ANSWER

Major RSA Attacks:

- **(a) Factoring Attacks:** Mathematical attempts (GNFS) to factor modulus n. Prevented by using ≥ 2048-bit keys.
- **(b) Small Public Exponent & Message Attacks:** If $e=3$ and message M is small ($M^3 < n$), $C = M^3$ without modulus reduction, so $M = \sqrt[3]{C}$ in plain arithmetic.
- **(c) Common Modulus Attack:** If same n is shared among users with different exponents e_1, e_2 where $\gcd(e_1, e_2)=1$, an eavesdropper can recover M from C_1, C_2 using Extended Euclidean Algorithm.
- **(d) Timing & Side-Channel Attacks:** Measuring hardware execution times or power draw during modular exponentiation reveals private key bits d. Mitigation: Constant-time implementations and cryptographic blinding ($C' = C \cdot r^e \pmod n$).
- **(e) Chosen-Ciphertext Attacks (CCA):** Exploits RSA's multiplicative homomorphic property $E(M_1) \cdot E(M_2) = E(M_1 \cdot M_2)$.

Need for Padding (OAEP): Textbooks RSA is deterministic (M always produces C). **Optimal Asymmetrical Encryption Padding (OAEP)** adds random bits and cryptographic hashing before encryption, making RSA probabilistic and semantically secure against CCA.